HANDBUCH ZUM VORTRAG · STAND 10. NOVEMBER 2026

Vom Token zur Entscheidung

Alle Begriffe und Entscheidungsregeln des Vortrags zum Nachschlagen: wie ein LLM funktioniert, was daraus folgt, und wie man GitHub Copilot in VS Code wirksam und sparsam einsetzt. Fachbegriffe stehen so da, wie man sie in Doku und Tools findet: auf Englisch.

1 Grundbegriffe
2 Neural Network und Training
3 Von Text zu Vorhersage
4 Wissen, Context, Hallucination
5 Systeme um das Modell
6 Was aus der Mechanik folgt
7 Modellwahl
8 Kosten und AI Credits
9 Deutsch oder Englisch
10 Context Engineering
11 Prompting
12 Copilot anpassen: sieben Bausteine
13 Autonomie und Kontrolle
14 Sicherheit und Verification
15 Zehn Regeln
16 Quellen und Stand

01 Wie ein LLM funktioniert

1 Grundbegriffe

WAS IST WAS

Artificial Intelligence (AI) — Sammelbegriff für Systeme, die Aufgaben lösen, für die man Intelligenz erwarten würde. Sagt nichts über die Technik.

Machine Learning (ML) — Die Regeln werden nicht programmiert, sondern von einem Suchverfahren aus Beispielen ermittelt. Ergebnis ist ein **Model**: eine Funktion mit gelernten Zahlen.

Deep Learning — Machine Learning mit vielschichtigen Neural Networks. Grundlage aller heutigen Sprach- und Bildmodelle.

Generative AI — Modelle, die neue Inhalte erzeugen (Text, Code, Bilder, Audio) statt nur zu klassifizieren.

Large Language Model (LLM) — Ein sehr großes Neural Network in Transformer-Architektur, trainiert darauf, das nächste Token eines Textes vorherzusagen. Beispiele: Claude, GPT, Gemini.

Supervised Learning — Training mit Eingaben und von Menschen vorgegebenen Soll-Ausgaben (Labels).

Self-Supervised Learning — Das Soll wird aus den Daten selbst gebildet: nächstes Token verdecken und vorhersagen. Kein Labeling nötig, deshalb praktisch unbegrenzt Trainingsmaterial. So werden LLMs trainiert.

Parameter — Die gelernten Zahlen eines Modells: **Weights** und **Biases**. Ein aktuelles großes LLM hat größenordnungsmäßig 10^{11} davon.

Hyperparameter — Einstellungen, die ein Mensch vor dem Training festlegt und die nicht gelernt werden: Anzahl Layers, Learning Rate, Batch Size.

2 Neural Network und Training

WIE DIE PARAMETER ENTSTEHEN

Neuron — Die kleinste Einheit: bildet eine gewichtete Summe seiner Eingaben, addiert einen Bias und wendet eine Activation Function an.

$$z = w_1x_1 + w_2x_2 + b \qquad \hat{y} = \sigma(z)$$

Weight — Faktor, mit dem eine Eingabe multipliziert wird. Bestimmt, wie stark sie zählt.

Bias — Konstante, die zur Summe addiert wird. Verschiebt die Schwelle des Neurons.

Activation Function — Nichtlineare Funktion am Ausgang des Neurons (z. B. Sigmoid, ReLU). Ohne sie ließe sich jedes tiefe Netz zu einer einzigen linearen Schicht zusammenrechnen.

Layer — Eine Schicht von Neuronen. Frühe Layers erkennen einfache Muster, spätere kombinieren sie.

Forward Pass — Eine Eingabe läuft einmal durch alle Layers; am Ende steht die Vorhersage.

Loss — Eine einzige Zahl, die sagt, wie falsch die Vorhersage war. Sie ist die vollständige Spezifikation des Trainingsziels: Was der Loss nicht misst, wird nicht gezielt gelernt.

Cross-Entropy Loss — Der Loss für LLMs: −log der Wahrscheinlichkeit, die das Modell dem tatsächlich folgenden Token gegeben hat. Bestraft hart, wenn das Richtige für fast unmöglich gehalten wurde.

Gradient — Die Steigung des Loss bezüglich eines Parameters: Wie ändert sich der Fehler, wenn ich diesen Parameter leicht erhöhe?

Gradient Descent — Das Lernverfahren: jeden Parameter ein kleines Stück gegen seinen Gradient verschieben, sodass der Loss sinkt. Milliardenfach wiederholt.

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

Learning Rate (η) — Die Schrittweite. Zu klein: Training kriecht. Zu groß: Jeder Schritt springt über das Minimum, das Training divergiert.

Batch / Stochastic Gradient Descent — Der Gradient wird nicht aus allen Daten berechnet, sondern aus einer kleinen zufälligen Stichprobe (Batch) geschätzt.

Backpropagation — Der Algorithmus, der in einem einzigen Rückwärtsdurchlauf (Backward Pass) per Chain Rule die Gradients **aller** Parameter berechnet. Backpropagation **berechnet**, Gradient Descent **ändert** – zwei verschiedene Vorgänge.

Overfitting / Generalization — Ein Modell kann Trainingsdaten auswendig lernen und bei Neuem versagen. Entscheidend ist das Verhalten auf ungesehenen Daten (Test Set).

Pretraining — Die lange, teure Phase: Next-Token Prediction auf riesigen Textmengen. Hier entsteht das Wissen.

Fine-Tuning — Nachtrainieren auf kleineren, gezielten Daten. Bei Chat-Modellen: das Frage-Antwort-Format.

RLHF (Reinforcement Learning from Human Feedback) — Nachtrainieren anhand menschlicher Bewertungen. Prägt Ton, Hilfsbereitschaft, Ablehnungsverhalten – und eine Neigung zuzustimmen.

Training Cutoff — Der Stichtag, an dem die Trainingsdaten enden. Alles danach kennt das Modell nur, wenn es im Context steht.

Inference — Die Nutzung des fertig trainierten Modells. Die Parameter sind eingefroren: Nichts, was ihr tippt, verändert das Modell.

> → FOLGE FÜR DIE PRAXIS
> Ob ein Modell eine Aufgabe beherrscht, steht in keiner Spezifikation. Das zeigt nur ein Test mit bekannten Antworten.

3 Von Text zu Vorhersage

WAS BEI JEDEM EINZELNEN TOKEN PASSIERT

Token — Ein Textstück aus einem festen Vocabulary: ein Wort, ein Wortteil, ein Satzzeichen. Das Modell sieht ausschließlich Token-IDs, keine Buchstaben.

Tokenizer / Byte-Pair Encoding (BPE) — Zerlegt Text in Tokens. Das Vocabulary entsteht vor dem Training, indem häufige Zeichenfolgen schrittweise verschmolzen werden. Häufiges wird ein Token, Seltenes zerfällt in viele.

Vocabulary — Die Menge aller Tokens eines Modells, typischerweise 100.000–200.000 Einträge.

Embedding — Der Vektor (einige tausend Zahlen), der ein Token repräsentiert. Training ordnet die Vektoren so an, dass ähnlich verwendete Tokens nah beieinander liegen. Grundlage von Semantic Search und RAG.

Attention — Der Mechanismus, mit dem jedes Token Information von anderen Tokens holt. Jede Position stellt eine Query, bietet einen Key und liefert einen Value; das Ergebnis ist eine gewichtete Mischung.

$$\mathrm{softmax}(QK^{\top} / \sqrt{d_k}) \cdot V$$

Aufwand wächst quadratisch mit der Länge.

Transformer — Die Architektur aller aktuellen LLMs: Dutzende gestapelte Blöcke aus Multi-Head Attention (bewegt Information *zwischen* Positionen) und Feed-Forward Network (verarbeitet Information *an* einer Position).

Logits — Die Rohwerte am Ende des Forward Pass: eine Zahl pro Token im Vocabulary.

Softmax — Wandelt Logits in Wahrscheinlichkeiten um, die sich zu 100 % summieren.

Sampling — Das nächste Token wird aus dieser Verteilung **gezogen**, nicht einfach das wahrscheinlichste genommen. Deshalb liefert dieselbe Anfrage verschiedene Antworten.

Temperature — Teilt die Logits vor dem Softmax. $T < 1$ schärft die Verteilung, $T > 1$ flacht sie ab und gibt Unwahrscheinlichem mehr Gewicht.

Top-p — Beschränkt das Ziehen auf die kleinste Menge von Tokens, deren Wahrscheinlichkeiten zusammen p ergeben. Schneidet den langen Rest ab.

Autoregression — Ein Token pro Forward Pass. Es wird an den Context angehängt, dann läuft alles erneut. Folgen: Output ist teuer, frühe Fehler pflanzen sich fort, und erzeugter Text ist von zugeliefertem nicht zu unterscheiden.

Reasoning / Thinking — Das Modell erzeugt zuerst Zwischenschritte (ein Notizblatt) und liest sie per Attention wieder ein, bevor es antwortet. Hilft bei mehrstufigen Problemen.

Reasoning Effort / Thinking Effort — Einstellung pro Aufgabe, wie viele Thinking Tokens das Modell aufwenden darf. Thinking Tokens werden als Output abgerechnet und sind oft nicht sichtbar.

4 Wissen, Context, Hallucination

WAS DAS MODELL WEISS – UND WAS NICHT

Zwei Wissensquellen

PARAMETER	CONTEXT
Im Training gelernt, danach eingefroren	In diesem Request mitgeschickt
Endet am Training Cutoff	Aktuell und exakt
Kennt nichts Internes	Begrenzt, kostet Tokens
Unscharf, verdichtet	Nach dem Request weg

Memory gibt es im Modell nicht. Was wie Erinnerung wirkt, ist Text, den der Harness erneut in den Context legt.

Context Window — Die maximale Eingabe in Tokens. Der Platz wird geteilt: System Prompt, Tool-Definitionen, Instructions, Files, History und euer Prompt konkurrieren. Der gesamte Inhalt wird bei **jedem** Request neu gesendet.

Lost in the Middle — Je voller das Fenster, desto unzuverlässiger wird Material gefunden, am schwächsten in der Mitte. Annehmen heißt nicht verwenden.

Compaction — Nähert sich der Context der Grenze, ersetzt der Harness die History durch eine Zusammenfassung. Details gehen dabei verloren. In der Copilot CLI startet das bei rund 80 %; die Schwellen variieren je Client.

Hallucination — Flüssige, aber falsche Ausgabe. Kein Schritt des Verfahrens prüft eine Aussage gegen die Wirklichkeit. Es ist derselbe Mechanismus, der richtige Antworten erzeugt, angewendet dort, wo die Trainingsdaten ihn nicht festgelegt haben. Gegenmittel sind strukturell: die Quelle in den Context legen, Tools rechnen und nachschlagen lassen, Belege verlangen und prüfen.

5 Systeme um das Modell

VOM MODELL ZUM AGENT

Harness — Das Programm um das Modell: stellt den Context zusammen, liefert Tool-Definitionen, führt Tool Calls aus, entscheidet über Abbruch und Compaction. GitHub Copilot in VS Code ist ein Harness, Claude Code ein anderer. Dasselbe Modell verhält sich in verschiedenen Harnesses unterschiedlich.

System Prompt — Die Anweisungen des Harness-Herstellers, die vor allem anderen im Context stehen.

Tool — Eine Fähigkeit außerhalb von Text: Datei lesen, Terminalbefehl, Websuche, API-Aufruf. Beschrieben durch Name, Beschreibung und Parameter-Schema.

Tool Call — Das Modell gibt strukturierten Text aus (Tool-Name plus Argumente). Der Harness prüft die Permission, führt aus und hängt das **Tool Result** an den Context. Das Modell führt nie selbst etwas aus.

Agentic Loop — Beobachten, entscheiden, handeln, Ergebnis lesen, wiederholen – bis die Aufgabe erledigt ist. Zuverlässig wird das vor allem durch Rückmeldung in der Schleife (Tests, Build).

Agent — Ein Modell in einem Harness mit Tools und einer Agentic Loop. Kann handeln, nicht nur antworten.

Subagent — Ein Agent, den der Haupt-Agent für eine Teilaufgabe startet. Arbeitet in einem **eigenen Context** und liefert nur das Ergebnis zurück. Hält den Haupt-Context sauber.

RAG (Retrieval-Augmented Generation) — Zu einer Frage werden passende Textstellen gesucht (meist über Embeddings) und in den Context gelegt. Wirksamstes Einzelmittel gegen Hallucination. Fehler sind dann meist Suchfehler: prüfen, was abgerufen wurde.

MCP (Model Context Protocol) — Offener Standard, über den ein Harness externe Tools und Daten anbindet. Ein **MCP Server** ist ein Paket aus Tools.

Prompt Injection — Text in einem Dokument, einer Webseite oder einem Tool Result, der als Anweisung formuliert ist. Funktioniert, weil Anweisungen und Daten denselben Kanal benutzen.

6 Was aus der Mechanik folgt

KANN UND KANN NICHT

MECHANISMUS	STÄRKE	GRENZE
Aus Beispielen gelernt	Muster, Sprache, Stil, Code-Idiome	Können zeigt nur der Test, nicht die Spezifikation
Next-Token Prediction	Flüssige, plausible Texte in Sekunden	Plausibel ist nicht wahr; nichts wird geprüft
Tokens statt Buchstaben	Jede Sprache, jedes Format	Zählen, exaktes Rechnen, Zeichenarbeit
Sampling	Varianten, Kreativität	Nicht deterministisch, nicht exakt reproduzierbar
Feste Parameter	Breites Weltwissen sofort verfügbar	Kein Lernen im Betrieb; nichts Aktuelles, nichts Internes
Begrenzter Context	Arbeitet exakt mit dem, was drinsteht	Vergisst, was herausfällt; verliert die Mitte
Harness und Tools	Kann lesen, schreiben, ausführen	Handelt auch auf falscher Grundlage

02 Wie man es richtig einsetzt

7 Modellwahl

DAS KLEINSTE MODELL, DAS DIE AUFGABE SCHAFFT

KATEGORIE	BEISPIELE	INPUT	CACHED	OUTPUT	GEEIGNET FÜR
Lightweight	Claude Haiku 5.5 GPT-6 Luna	$0.10 $0.10	$0.01 $0.01	$0.50 $0.50	Umformatieren, Boilerplate, Commit Messages, einfache Fragen
Versatile	Claude Sonnet 5.5 GPT-6.1 Terra	$2.00 $1.00	$0.10 $0.05	$10.00 $5.00	Der Alltag: Coding, Agent Mode, Review, Tests, Doku
Powerful	Claude Opus 5.5 GPT-6.1 Sol Claude Fable 5.2	$4.00 $2.00 $8.00	$0.20 $0.10 $0.20	$20.00 $10.00 $40.00	Architektur, Debugging über viele Dateien, lange autonome Aufgaben

US$ je 1 Mio. Tokens, Listenpreise und Kategorien laut GitHub Docs, abgerufen am 10.11.2026. Auswahl; welche Modelle freigeschaltet sind, entscheidet der Admin. Preise ändern sich – vor Entscheidungen die Quelle prüfen (Abschnitt 16).

Vorgehen

1. **Start mit Versatile** oder **Auto** (wählt je Request; wo angeboten, mit „Optimize for“ Efficiency, Balance oder Intelligence).
2. **Runter auf Lightweight**, wenn die Aufgabe mechanisch ist und das Ergebnis sofort sichtbar.
3. **Hoch auf Powerful**, wenn mehrstufig, ein früher Fehler teuer – oder zwei Versuche gescheitert sind.
4. **Thinking Effort** nur erhöhen, wenn wirklich geschlussfolgert werden muss. Bei Nachschlagen und Umformatieren kostet es nur.

Benchmarks und eigene Eval

Benchmarks taugen als erster Filter, nicht als Entscheidung: Kein Leaderboard misst eure Aufgabe, Testbedingungen variieren, und veröffentlichte Tests landen in späteren Trainingsdaten.

Mini-Eval (ein halber Tag): 5–10 echte Aufgaben mit bekanntem Sollergebnis. Kriterien vorher festlegen. Jede Aufgabe dreimal laufen lassen – ein einzelner Run misst den Zufall. Kosten und Dauer mit notieren.

8 Kosten und AI Credits

ABGERECHNET WIRD IN TOKENS

FAKT	WERT
1 AI Credit	0,01 US$
Copilot Enterprise	3.900 Credits pro User und Monat
Copilot Business	1.900 Credits pro User und Monat
Pooling	Über alle Lizenzen der Abrechnungseinheit; kein Übertrag, Reset am 1. des Monats (UTC)
Gratis	Inline Completions und Next Edit Suggestions
Verbraucht Credits	Chat, Agent, Copilot CLI, Cloud Agent, Spaces
Pool leer	Kostenpflichtige Weiternutzung (Default an) oder Sperre bis Monatswechsel, je nach Policy
Budgets	Pro User, Cost Center, Organisation, Enterprise

Drei Token-Arten

Input — alles, was gesendet wird: der gesamte Context, bei jedem Request.

Cached Input — ein unveränderter Anfang des Context, den der Anbieter schon verarbeitet hat. Kostet etwa ein Zehntel bis ein Zwanzigstel. Stabile Instructions sparen also Geld; ein wechselnder Prefix zerstört den Cache.

Output — alles Erzeugte, inklusive Thinking Tokens. Kostet typischerweise das Fünffache des Inputs.

> **Rechenbeispiel** (Sonnet 5.5, Start-Context 30K, 20 Agent-Steps, +4K je Step, 1K Output je Step): 1,55 Mio. Input-Tokens, rund **59 Credits** mit Caching, rund **330** ohne. Bei 40 Steps: 134 Credits.

Was Credits frisst

- Lange Sessions: Die History reist jedes Mal mit, die Input-Tokens wachsen quadratisch
- Große Files und Tool Results im Context
- Agent Loops mit vielen Steps
- Hoher Thinking Effort für Routine
- Powerful-Modell für Trivialaufgaben
- Retries, weil die Aufgabe unklar war

Was hilft

- Neuer Chat pro Aufgabe
- Gezielte Files statt ganzer Ordner
- Modell passend zur Aufgabe
- Präziser Prompt: ein Run statt drei
- Subagents für Recherche (eigener Context)
- Kurze, stabile Instructions (Cache bleibt warm)

> → FOLGE FÜR DIE PRAXIS
> Die Modellkosten sind meist der kleinere Posten. Zwanzig Minuten Review kosten mehr als der Run. Am günstigsten ist der Weg mit den wenigsten Korrekturschleifen.

9 Deutsch oder Englisch

KOSTEN MESSBAR, QUALITÄT DIFFERENZIERT

Kosten: Das Vocabulary ist auf Englisch und Code optimiert. Deutsch braucht für denselben Inhalt mehr Tokens – als Richtwert das 1,3- bis 1,5-Fache, bei Komposita mehr. Das wirkt auf Kosten und auf den Platz im Context Window. Der Faktor hängt vom Tokenizer ab: für eigene Texte messen, nicht schätzen. **Qualität:** Aktuelle große Modelle verstehen und schreiben Deutsch sehr gut. Der Abstand wächst bei kleinen Modellen, bei Fachterminologie und dort, wo Doku, Code und Fehlermeldungen ohnehin englisch sind.

WAS	EMPFEHLUNG
Instructions, Skills, Agents, Prompt Files	**Englisch.** Werden bei jedem Request gesendet; Begriffe sind eindeutig; teamübergreifend lesbar.
Code, Commits, Fachbegriffe	**Englisch.** Fachbegriffe nie übersetzen (Gradient Descent, nicht „Gradientenabstieg“).
Ad-hoc im Chat	Die Sprache, in der ihr am präzisesten formuliert. Präzision schlägt Sprache.
Ausgabe	Explizit festlegen: `Answer in German.`

10 Context Engineering

BEWUSST ENTSCHEIDEN, WAS DAS MODELL SIEHT

Was im Context steht (von oben nach unten)

1. System Prompt des Harness
2. Tool-Definitionen – Built-in und jeder MCP Server
3. Custom Instructions und Skill-Beschreibungen
4. Angehängte Files, geöffnete Datei, Selection
5. History und Tool Results (größter Posten in Agent-Sessions)
6. Euer Prompt – der kleinste Teil

In VS Code zeigt ein kleiner Kreis am Chat-Eingabefeld den Füllstand. In der Copilot CLI: `/context` und `/compact`.

Neuer Chat, wenn …

- die Aufgabe wechselt
- drei Korrekturen nichts gebracht haben (im Context stehen dann drei falsche Ansätze)
- mehrere Compactions hinter euch liegen
- ihr von Erkunden zu Umsetzen wechselt

Übergabenotiz: Lange Arbeit am Ende einer Session in fünf Zeilen zusammenfassen lassen (Ziel, Stand, Entscheidungen, offene Punkte, nächster Schritt) und die neue Session damit beginnen.

11 Prompting

SECHS BAUSTEINE, SECHS PATTERNS

BAUSTEIN	SCHWACH	BELASTBAR
Ziel	„Fix den Export“	„Der CSV-Export bricht ab 10.000 Zeilen ab. Ziel: läuft für 1 Mio. Zeilen durch.“
Hintergrund	*fehlt*	„Läuft nightly in CI mit 2 GB Speicherlimit. Streaming ist erlaubt.“
Constraints	„Mach es sauber“	„Public API nicht ändern. Keine neue Dependency.“
Output-Format	„Zeig mal“	„Erst Plan in fünf Punkten, dann Diff, dann offene Fragen.“
Acceptance Criteria	*fehlt*	„Bestehende Tests grün, plus ein neuer Test mit 1 Mio. Zeilen.“
Umgang mit Lücken	*fehlt*	„Wenn etwas unklar ist: fragen, nicht raten. Annahmen auflisten.“

Die beiden roten Bausteine fehlen am häufigsten und verändern das Ergebnis am stärksten.

Beispiele zeigen (Few-Shot) — Ein gutes Beispiel für den gewünschten Output schlägt drei Absätze Beschreibung.

Das Warum nennen — „Keine neue Dependency, weil das Audit sechs Wochen dauert“ generalisiert besser als ein nacktes Verbot.

Positiv formulieren — Sagen, was passieren soll. Aktuelle Modelle nehmen Anweisungen wörtlich.

Erst Plan, dann Ausführung — Der Plan kostet einen Bruchteil und zeigt Missverständnisse, bevor Code entsteht.

Verify-Loop mitgeben — Tests, Build, Linter, Typecheck: Ein Agent, der sein Ergebnis selbst prüfen kann, wird deutlich zuverlässiger.

Neu starten statt flicken — Nach zwei Fehlversuchen: neuer Chat mit besserem Prompt, nicht die fünfte Korrektur.

Prompting Guides — Anthropic, OpenAI und Google veröffentlichen eigene, modellspezifische Guides. Den Guide für das meistgenutzte Modell einmal lesen (Abschnitt 16).

12 Copilot anpassen: sieben Bausteine

ALLES DATEIEN IM REPO: VERSIONIERT, REVIEWBAR, FÜRS GANZE TEAM

BAUSTEIN	DATEI	WIRD GELADEN	WOFÜR

Custom Instructions	`.github/copilot-instructions.md`
	`.github/instructions/*.instructions.md`
	`AGENTS.md`
		automatisch, immer	Konventionen, Projektwissen

Prompt Files	`.github/prompts/*.prompt.md`	manuell, per `/name`	Wiederkehrende Anfrage
Agent Skills	`.github/skills/<name>/SKILL.md` `~/.copilot/skills/<name>/SKILL.md`	automatisch, wenn relevant	Verfahren mit Scripts und Vorlagen
Custom Agents	`.github/agents/<name>.agent.md`	manuell, Agent-Dropdown; oder als Subagent	Rolle mit eigenen Tools
MCP Servers	`mcp.json (z. B. .vscode/mcp.json)`	Tools stehen bereit	Externe Systeme und Daten
Hooks (Preview)	`.github/hooks/*.json`	deterministisch, an Lifecycle-Events	Garantien statt Bitten
Agent Plugins (Preview)	Marketplace	nach Installation	Bündel aus Skills, Agents, Hooks, MCP

Custom Instructions

`.github/copilot-instructions.md`

```
## Project
Order service. Java 21, Spring Boot 3, Gradle.

## Conventions
- Run `./gradlew test` before declaring a task
done.
- Never edit files under `generated/`.
- Public API changes need an ADR in `docs/adr/`.

## Language
- Answer in German. Code, comments, commits in
English.
```

`.github/instructions/sql.instructions.md`

```
---
applyTo: "**/*.sql"
---
Use explicit column lists. No `SELECT *`.
```

- Entscheidungen benennen, nicht guten Stil beschreiben
- Engster Scope, der noch alle Fälle erfasst
- Widersprüche entfernen – es gibt keine verlässliche Rangfolge
- Quartalsweise kürzen: Jede Zeile wird bei jedem Request bezahlt

Agent Skills

`.github/skills/ms-style/SKILL.md`

```
---
name: ms-style
description: Write or review user-facing docs
  following the Microsoft Writing Style Guide.
  Use for README files, how-to guides,
  release notes and UI text.
---
# Rules
- Sentence-style capitalization in headings.
- Second person, active voice, present tense.

Approved terms: references/word-list.md
After writing, run scripts/check_terms.py.
```

Progressive Disclosure: (1) `name` und `description` stehen immer im Context, rund 100 Tokens. (2) Die SKILL.md wird erst geladen, wenn das Modell den Skill für relevant hält. (3) `references/` und `scripts/` nur bei Bedarf.

Die **description** trägt die Hauptlast: Sie ist alles, was das Modell bei der Entscheidung sieht. Hineinschreiben, *was* der Skill tut und *wann* er greifen soll.

Eigenen Skill bauen – fünf Schritte

1. Aufgabe dreimal von Hand im Chat lösen
2. Was war jedes Mal gleich? Vom Agent in eine SKILL.md schreiben lassen
3. Alles Exakte in Scripts auslagern: rechnen, validieren, formatieren
4. description schärfen: wann greifen, wann nicht
5. An drei echten Fällen testen, dann per Pull Request ins Repo

Gute Kandidaten: Styleguides (Google developer documentation style guide, Microsoft Writing Style Guide), Design-Vorlagen (Corporate Design, Slide-Templates), reproduzierbare Abläufe (Release Notes, Incident Report, Migrations- und Review-Checklisten).

Agent Skills sind ein offener Standard: Derselbe Ordner funktioniert in Copilot, Claude Code und anderen Agents. Copilot liest auch `.claude/skills/` und `.agents/skills/`.

Custom Agents

`.github/agents/reviewer.agent.md`

```
---
name: Reviewer
description: Review code for incorrect behavior
  and missing tests without editing files
tools: ['read', 'search']
handoffs:
  - label: Fix findings
    agent: agent
    prompt: Fix the findings listed above.
---
Review the code supplied in the request.
Report findings with file and line references.
Do not edit files or run terminal commands.
```

tools begrenzt, was der Agent tun *kann*. **handoffs** verketten Rollen (Plan → Implement → Review). Anlegen über **Chat: New Custom Agent**.

Skill oder Agent? Skill = Wissen und Verfahren, das jeder Agent nutzen kann. Agent = Rolle mit eigenem Auftrag und eingeschränkten Tools.

MCP Servers

`.vscode/mcp.json`

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

- Ein MCP Server ist fremder Code mit euren Zugangsdaten: Herausgeber prüfen, Version pinnen, Credentials beschränken
- Tool-Beschreibungen sind Teil des Prompts: einmal lesen
- Jede Tool-Definition kostet bei jedem Request Context: nur aktivieren, was die Aufgabe braucht

Hooks

`.github/hooks/guard.json`

```
{
  "version": 1,
  "hooks": {
    "preToolUse": [{
      "type": "command",
      "command": "node .github/hooks/guard.cjs",
      "timeoutSec": 10
    }]
  }
}
```

Ein Hook ist ein Shell-Befehl, der deterministisch an einem Lifecycle-Event läuft. Er bekommt das Event als JSON auf stdin; ein `preToolUse`-Hook kann einen Tool Call mit `"permissionDecision": "deny"` ablehnen.

Events: `sessionStart`, `userPromptSubmitted`, `preToolUse`, `postToolUse`, `preCompact`, `subagentStart`, `subagentStop`, `agentStop`, `sessionEnd`. In VS Code noch Preview; Hooks ersetzen keine Sandbox und keine Org-Policy.

Prompt Files und Agent Plugins

Prompt File — eine gespeicherte Anfrage mit Platzhaltern unter `.github/prompts/`, aufgerufen per `/name` im Chat. Richtig, wenn dieselbe Anfrage regelmäßig wiederkommt, aber kein mehrstufiges Verfahren ist. **Agent Plugin** — ein installierbares Bündel aus Skills, Custom Agents, Hooks, MCP Servers und Slash Commands, verteilt über Marketplaces. Richtig, wenn ein Setup über Teams hinweg geteilt werden soll. Vor der Installation Herausgeber und Inhalt prüfen: Ein Plugin ist ausführbarer Code.

So wenig Struktur wie nötig

STUFE	AUFWAND	ANGEMESSEN, WENN …

Einzelne Anfrage im Chat	keiner	einmalig, oder das Ziel ist noch offen
Prompt mit sechs Bausteinen	Minuten	es darauf ankommt
Prompt File	unter einer Stunde	dieselbe Anfrage wiederkommt
Custom Instructions	eine Stunde plus Pflege	eine Regel für viele Aufgaben gilt
Skill	halber Tag plus Pflege	ein mehrstufiges Verfahren regelmäßig läuft
Custom Agent, MCP, Hooks	Einrichtung und Security-Review	gehandelt werden muss und Garantien nötig sind
Agent Plugin	Paketierung	das Setup teamübergreifend geteilt werden soll
Klassische Software	oft kleiner als gedacht	das Verfahren exakt ist und die Eingaben strukturiert sind

13 Autonomie und Kontrolle

ASK, PLAN, AGENT – UND WAS WIRKLICH GARANTIERT IST

AGENT	IHR BEURTEILT	GEEIGNET FÜR
Ask	jede Antwort	Verstehen, Optionen abwägen
Plan	den Plan	Größeres und Unklares
Agent	Aktionen – oder nur das Ergebnis	Umsetzen; mit Einzelfreigabe interaktiv, mit Auto-Approve autonom

Fünf Fragen vor mehr Autonomie: Ist die Aufgabe eindeutig? Sind die Aktionen umkehrbar? Was kostet ein Fehler? Lässt sich Erfolg automatisch prüfen? Worauf haben die Tools Zugriff?

Mit der geringsten Autonomie beginnen, die die Aufgabe erfüllt. Jeder erste Durchlauf interaktiv – auch nach einem Modellwechsel.

ART DER KONTROLLE	BESTIMMT, WAS …
Guidance Instructions, Prompts, Skills	**wahrscheinlich** passiert
Orchestration Agents, Handoffs, Plan-Schritt	**üblicherweise** passiert
Enforcement Tool Approval, Hooks, Permissions, Sandbox	**möglich** ist

> „Ändere nichts unter `prod/` “ in einer Instruction kann verdrängt werden. Ein fehlendes Schreibrecht oder ein `preToolUse` -Hook mit `deny` nicht. Alles Teure und Unumkehrbare gehört in die dritte Zeile.

14 Sicherheit und Verification

PRÜFEN IST EUER JOB – MACHT IHN EUCH LEICHT

Prompt Injection

Im Context ist alles Text. Das Modell kann nicht sicher unterscheiden, ob ein Satz von euch stammt oder aus einem README, Issue, PDF oder Tool Result. Das ist eine Eigenschaft der Architektur; eine allgemeine Lösung gibt es derzeit nicht.

> **Gefährliche Dreierkombination:** vertrauliche Daten + nicht vertrauenswürdiger Inhalt + ausgehender Kanal. Treffen alle drei zusammen, kann ein Angreifer Daten abziehen. Eines der drei entfernen.

Gehört nicht in den Prompt: gültige Secrets, Tokens, Passwörter; personenbezogene Daten identifizierbarer Personen; Vertrauliches von Dritten. Gelangt ein Secret in einen Chat: rotieren.

Tool Approvals nicht reflexhaft auf „Always allow" stellen, vor allem nicht für Terminal und Netzwerk.

Verification

Vorher: Acceptance Criteria festlegen, bevor die Arbeit beginnt. **Währenddessen:** dem Agent einen Verify-Loop geben. **Danach:** Prüfbarkeit einfordern – Diff statt Prosa, Quelle zu jeder Aussage, Liste der Annahmen.

WOHIN GEHT DAS ERGEBNIS?	PRÜFTIEFE
Erkunden, Prototyp	Durchsicht
Eigene Meinungsbildung	Stichproben
Pull Request, Kollegen	Jede Änderung gelesen und verstanden
Production, Vertrag, extern	Vollständig; Zahlen unabhängig nachgerechnet

Besondere Sorgfalt: alles Unumkehrbare (Senden, Deployen, Löschen), Schreibzugriff auf Geteiltes, erste Durchläufe nach Modell- oder Konfigurationswechsel.

15 Zehn Regeln

FÜR MORGEN FRÜH

01 Das Modell sagt Tokens vorher. **Geprüft wird nichts** – außer von euch.

02 Was nicht im **Context** steht, existiert nicht.

03 **Neuer Chat** pro Aufgabe.

04 Das **kleinste Modell**, das die Aufgabe schafft.

05 Ziel, Constraints, **Acceptance Criteria** – vor dem Start.

06 Erst **Plan**, dann Agent.

07 Gebt dem Agent einen **Verify-Loop**.

08 Was sich wiederholt, wird ein **Skill**.

09 Instructions bitten, **Hooks und Permissions** garantieren.

10 Dauerhaftes auf **Englisch**: kurz und widerspruchsfrei.

16 Quellen und Stand

ALLES PRODUKTBEZOGENE ÄNDERT SICH – HIER NACHSEHEN

Offizielle Doku (abgerufen am 10.11.2026)

- GitHub Docs: *Usage-based billing for organizations and enterprises*
- GitHub Docs: *Models and pricing for GitHub Copilot*
- GitHub Docs: *Supported AI models in Copilot*
- GitHub Docs: *Customization cheat sheet*
- GitHub Docs: *Context management*
- VS Code Docs: *Customization* – Custom Agents, Agent Skills, Hooks, Agent Plugins, Language Models
- modelcontextprotocol.io · agentskills.io

Zum Vertiefen

- **3Blue1Brown**, Neural Networks und Transformers – die anschaulichste visuelle Erklärung
- **Andrej Karpathy**, *Deep Dive into LLMs like ChatGPT*
- **Anthropic**, *Building effective agents* und der Prompt Engineering Guide
- **OpenAI** und **Google**: die jeweiligen Prompting Guides
- **Simon Willison**, Sammlung zu Prompt Injection
- **OWASP** Top 10 for LLM Applications

Hinweise zur Genauigkeit. Preise, Credits, Modellnamen und Dateipfade stammen aus den genannten offiziellen Seiten vom 10.11.2026. Der Faktor 1,3–1,5 für deutsche Texte ist ein Richtwert, keine Messung an unseren Texten. Die Zerlegungen der Tokenizer-Demo sowie die Attention-Gewichte und Logits im Vortrag sind illustrativ. Vier Fragen kann nur unsere Administration beantworten: welche Modelle freigeschaltet sind, ob kostenpflichtige Weiternutzung erlaubt ist, welche Budgets gelten und welche MCP Servers und Plugins zugelassen sind.